Resumo de Estudo e Apresentação

Assistente de Receitas — Projeto Final IA 2 (Gemini + Function Calling + APIs Externas + Front-end MVC)
Integrantes: Matheus Figueira Merlos, João Pedro Marucci Pagluiso e Gustavo Scandelae de Flores

1. O Projeto em 1 Frase (Abertura da Apresentação)

> **Frase para decorar:** "O usuário informa os ingredientes que tem em casa no front-end; o nosso backend em Flask (organizado em MVC) valida os dados antes de gastar tokens e envia ao Gemini; o Gemini usa **Function Calling** para pedir funções que consultam a **TheMealDB** (receitas reais) e a **TheCocktailDB** (bebida), e devolve em JSON uma receita real traduzida para português com foto."

Fluxo Completo em 8 Passos

1. **Front-end (`script.js`)**: Usuário adiciona ingredientes (validados no navegador por `ingredienteValido()`), escolhe o tipo de refeição e bebida, e clica em **Criar Receita**.
2. **Requisição HTTP**: O navegador envia `POST /api/receitas/gerar` com JSON `{ingredientes, tipoRefeicao, bebida, evitar}`.
3. **Controller (`receita_controller.py`)**: Valida os ingredientes no Python (`validar_formato_ingredientes`). Se houver números (`1+1`), frases ou palavras fora do tema, devolve **HTTP 400 (gasto = 0 tokens)**.
4. **Model (`receita_model.py`)**: Envia o pedido ao Gemini junto com a `INSTRUCAO` (System Instruction) e as 3 funções declaradas em `tools`.
5. **1º Function Call (Busca)**: O Gemini traduz os ingredientes para inglês (ex.: `["rice", "chicken"]`) e pede `buscar_receitas_por_ingredientes`. O Python consulta a TheMealDB, calcula a interseção, filtra sobremesas e devolve a lista ao Gemini.
6. **2º Function Call (Detalhes + Bebida)**: O Gemini escolhe uma receita da lista e pede `ver_receita(id_receita)` (e `sugerir_bebida()`, se o usuário marcou). O Python consulta as APIs e devolve os dados reais.
7. **Resposta Final e Anti-Alucinação**: O Gemini traduz tudo para português em JSON. O Python valida se `ver_receita` realmente rodou, normaliza os campos e injeta a `foto` e o `idReceita` vindos direto da API.
8. **View (`resposta_view.py`) e Tela**: O backend responde HTTP 200 com `{sucesso, dados, funcoes}` e o front desenha o cartão usando `textContent` (seguro contra XSS).

2. Bloqueio de Tokens Fora do Tema (Ex: `1+1`, `carro`, perguntas)

> **Pergunta clássica do professor:** "Se eu digitar `1+1` ou pedir outra coisa fora do tema, ele vai rodar o código todo e gastar tokens do Gemini?"
> **Resposta:** Não! Implementamos **3 Barreiras de Proteção em cascata** que bloqueiam entradas fora do tema com **zero tokens gastos** nas duas primeiras camadas.

Barreira	Onde fica no código	O que ela bloqueia	Gasto de Tokens
1ª Barreira (Front-end JS)	`ingredienteValido()` e `PALAVRAS_BLOQUEADAS` em `frontend/script.js`	Bloqueia números (`1+1` , `123`), símbolos (`+` , `=` , `?`), frases com mais de 4 palavras, palavras sem vogal (`bcdfg`), letras repetidas (`aaaa`) e palavras de perguntas/programação/objetos (`quanto` , `calcule` , `carro` , `tijolo` , `piada` , `python`) na hora de clicar em Adicionar.	**0 tokens** (nem faz requisição ao servidor)
2ª Barreira (Back-end Python)	`_item_invalido()` e `validar_formato_ingredientes()` em `receita_model.py`	Executa a mesma validação no servidor Python antes de chamar o Gemini. Se alguém mandar `1+1` ou `"quanto e um mais um"` direto pelo Postman/Insomnia, o Controller retorna **HTTP 400** imediatamente.	**0 tokens** (barra antes do Gemini)
3ª Barreira (System Prompt)	Passo 1 da `INSTRUCAO` em `receita_model.py`	Se passar alguma palavra em português que não estava na lista mas não é comida (ex.: `"travesseiro"` , `"bicicleta"`), o Gemini é proibido de chamar funções e responde na hora `{"ingredientesValidos": false}` , encerrando na 1ª volta com **HTTP 422**.	**Mínimo possível** (não roda o loop nem chama APIs)

3. Arquitetura MVC e Papel de Cada Arquivo

Arquivo	Camada MVC	Responsabilidade
`backend/app.py`	Ponto de Entrada	Cria o servidor Flask, ativa o `CORS`, registra o Blueprint de rotas, trata erros 404/405 em JSON e sobe em `127.0.0.1:5000`.
`backend/controllers/receita_controller.py`	**Controller**	Rotas `GET /api/status` e `POST /api/receitas/gerar`. Recebe o JSON, aciona a validação inicial (barreira de 0 tokens), chama o Model e devolve pela View.
`backend/models/receita_model.py`	**Model**	Configuração do Gemini, System Instruction, validação anti-tokens, fallback de modelos e o loop manual de Function Calling.
`backend/models/ferramentas_model.py`	**Model (Tools)**	As 3 funções Python que o Gemini pode pedir (`buscar_receitas_por_ingredientes`, `ver_receita`, `sugerir_bebida`) que consultam TheMealDB e TheCocktailDB.
`backend/views/resposta_view.py`	**View**	Padroniza todas as respostas JSON da API (`sucesso`, `erro`, `status`).
`frontend/index.html`, `style.css`, `script.js`	**Front-end** (Interface)	Interface do usuário separada do backend: formulário de tags, validação no navegador, chamada `fetch` com `AbortController` e receitas salvas no `localStorage`.

4. Function Calling e Busca de Receitas em Detalhe

- **O que é Function Calling:** O Gemini **não** acessa a internet e **não** executa funções sozinho. Nós passamos a lista de funções em `tools=list(FUNCOES.values())`. O SDK lê o **nome**, os **type hints** (`ingredientes: list[str]`) e a **docstring** de cada função. Quando precisa de dados, o Gemini devolve um pedido `function_call`; o nosso código Python executa a função real em `executar_funcoes()` e devolve o resultado com `Part.from_function_response`.
- **Por que `automatic_function_calling` está desligado (`disable=True`)?** Para controlarmos o loop manualmente (até 6 voltas), registrarmos os nomes das funções chamadas (mostrando no terminal e no rodapé do card), filtrarmos receitas já vistas (`evitar`) no código e garantirmos que `ver_receita` foi chamada.
- **Como resolvemos o "Bug do Arroz Doce":**
  1. A TheMealDB gratuita só filtra 1 ingrediente por vez (`filter.php?i=rice`). Nossa função consulta cada ingrediente separadamente e calcula a **interseção** (quantos ingredientes do usuário cada prato usa).
  2. Consultamos a categoria `Dessert`: se o usuário pediu *Sobremesa*, mantemos apenas sobremesas; para qualquer outro tipo (*Almoço/Jantar, Café da Manhã, Lanche*), **removemos todas as**

sobremesas.

3. Se nenhuma receita usar todos os ingredientes (caso parcial), ordenamos pela **raridade** (`1 / total_receitas_do_ingrediente`) e mantemos as receitas do ingrediente principal (mais raro).
4. Resolvemos singular/plural (`tomato` / `tomatoes`) e nomes compostos (`chocolate` → `dark_chocolate`) usando a lista oficial da API em cache (`_nomes_cache`).

5. Números, Limites e Códigos HTTP para Decorar

Item / Configuração	Valor no Projeto	Explicação Rápida
Modelos do Gemini	`gemini-3.1-flash-lite` (principal) `gemini-3.8-flash` (reserva)	O principal é mais rápido e estável; se der erro (503/429/timeout), o `try/except` em `chamar_gemini` chama o reserva automaticamente.
Timeouts (Limites de tempo)	**10 s** (APIs externas) | **30 s** (cada chamada Gemini) **100 s** (loop total backend) | **150 s** (front-end)	Evita que o servidor ou o navegador fiquem travados esperando para sempre.
Limites de Validação e Loop	Máx. **6 voltas** no loop | Máx. **20 ingredientes** **2 a 35 caracteres** e máx. **4 palavras** por item	Protege a cota de tokens e impede loops infinitos ou textos longos no prompt.
Códigos HTTP de Erro	`400` (Formato/Fora do tema - 0 tokens) `404` (Sem receita / Rota inexistente) `405` (Método errado) | `422` (Item não é alimento) `429` (Cota Gemini) | `500` (Erro interno/chave) `502` (Sem `ver_receita`) | `503` (Sobrecarga) | `504` (>100s)	Cada situação tem um código HTTP semântico e uma mensagem clara em português.

6. Todos os Códigos do Projeto (Para Estudar Sem Abrir o Editor)

Abaixo estão os arquivos completos do projeto com comentários nos pontos principais para todo o grupo estudar direto por este PDF:

1. backend/app.py (Servidor Flask, CORS e Rotas de Erro)

```
import os
from dotenv import load_dotenv
from flask import Flask
from flask_cors import CORS
from controllers.receita_controller import receita_bp
from views import resposta_view

# Carrega as variaveis do arquivo .env (como GEMINI_API_KEY e PORT)
load_dotenv()

# Cria a aplicacao Flask (apontando para a pasta frontend) e libera o CORS
app = Flask(__name__, static_folder="../frontend", static_url_path="")
CORS(app)

# Registra o Blueprint com as rotas /api/status e /api/receitas/gerar
app.register_blueprint(receita_bp)


# Abre a tela do front-end (index.html) ao acessar http://127.0.0.1:5000/
@app.route("/")
def pagina_inicial():
    return app.send_static_file("index.html")


# Em vez de devolver erro 404/405 em HTML, devolve JSON padronizado pela View
@app.errorhandler(404)
def rota_inexistente(_erro):
    return resposta_view.erro("Rota nao encontrada.", 404)


@app.errorhandler(405)
def metodo_nao_permitido(_erro):
    return resposta_view.erro("Metodo nao permitido para esta rota.", 405)


porta = int(os.getenv("PORT", 5000))

if __name__ == "__main__":
    print(f"Servidor Assistente de Receitas rodando na porta {porta}...", flush=True)
    # 127.0.0.1: so este computador acessa. Com debug=True, 0.0.0.0 abriria o depurador do Flask para a rede
    app.run(host="127.0.0.1", port=porta, debug=True)
```

2. backend/controllers/receita_controller.py (Controller: Porta de Entrada e Validação)

```
from flask import Blueprint, request
from models import receita_model
from views import resposta_view

# Blueprint agrupa as rotas de receita para registrar no app.py
receita_bp = Blueprint("receita", __name__)
```

```
@receita_bp.route("/api/status", methods=["GET"])
def verificar_status():
    # Devolve apenas True/False para indicar se a chave existe, sem expor a chave real
    return resposta_view.status(bool(receita_model.chave_gemini))


@receita_bp.route("/api/receitas/gerar", methods=["POST"])
def gerar_receita():
    try:
        # Lê o JSON enviado pelo front-end; se vier invalido, usa dicionario vazio
        dados = request.get_json(silent=True)
        if not isinstance(dados, dict):
            dados = {}

        # 1a Barreira (0 tokens): valida formato, numeros, frases e palavras fora do tema antes de chamar o
Gemini
        lista, mensagem_erro = receita_model.validar_formato_ingredientes(dados.get("ingredientes"))
        if mensagem_erro:
            return resposta_view.erro(mensagem_erro, 400)

        # Lista de permissao: garante que o tipo de refeicao seja um dos 4 validos
        tipo = dados.get("tipoRefeicao")
        if tipo not in receita_model.TIPOS_REFEICAO:
            tipo = receita_model.TIPOS_REFEICAO[0]

        # IDs de receitas ja vistas para o botao "Outra receita" nao repetir
        evitar = dados.get("evitar") if isinstance(dados.get("evitar"), list) else []
        evitar = [str(i) for i in evitar][:20]

        # Chama o Model (Gemini + Function Calling) e devolve a resposta formatada pela View
        receita, funcoes = receita_model.gerar_receita(lista, tipo, bool(dados.get("bebida")), evitar)
        return resposta_view.sucesso(receita, funcoes)

    except receita_model.ErroReceita as erro:
        # Erros conhecidos e tratados (ex: 422 item nao culinario, 404 sem receita, 429 limite da API)
        return resposta_view.erro(erro.mensagem, erro.codigo)
    except Exception as erro_geral:
        print(f"[Erro Geral] {erro_geral}", flush=True)
        return resposta_view.erro("Erro ao gerar a receita. Tente novamente.", 500)
```

3. backend/views/resposta_view.py (View: Padronização das Respostas JSON)

```
from flask import jsonify


def inicio(modelos):
    return jsonify({
        "servico": "API Assistente de Receitas",
        "status": "online",
        "modelos": modelos
    }), 200


# Padroniza a resposta de GET /api/status
def status(tem_chave):
    return jsonify({
        "status": "online",
        "servico": "API Assistente de Receitas - Projeto Final IA2",
        "temChaveConfigurada": tem_chave
    }), 200
```

```
# Padroniza a resposta de sucesso (HTTP 200) com a receita e as funcoes chamadas pelo Gemini
def sucesso(dados, funcoes=None):
    return jsonify({
        "sucesso": True,
        "dados": dados,
        "funcoes": funcoes or []
    }), 200


# Padroniza qualquer resposta de erro (HTTP 400, 404, 422, 429, 500, etc.)
def erro(mensagem, codigo=400):
    return jsonify({
        "sucesso": False,
        "erro": mensagem,
        "mensagem": mensagem
    }), codigo
```

4. backend/models/receita_model.py (Model Principal: Validação, Gemini e Loop de Function Calling)

```
import os
import re
import json
import time
from dotenv import load_dotenv
from google import genai
from google.genai import types
from models.ferramentas_model import FUNCOES

load_dotenv()

# Modelo principal rapido (responde melhor em horario de pico) e modelo reserva (fallback automatico)
MODELO_PRINCIPAL = "gemini-3.1-flash-lite"
MODELO_RESERVA = "gemini-3.8-flash"

TIPOS_REFEICAO = ["Almoco/Jantar", "Cafe da Manha", "Lanche", "Sobremesa"]
LIMITE_SEGUNDOS = 100  # tempo maximo de uma geracao (varias idas e voltas com o Gemini)

# Inicializa o cliente da API do Gemini com timeout de 30s por chamada
chave_gemini = os.getenv("GEMINI_API_KEY")
cliente = genai.Client(api_key=chave_gemini, http_options={"timeout": 30000}) if chave_gemini else None

# System Instruction: define o papel do assistente, as regras de validacao (2a barreira) e o formato JSON
INSTRUCAO = """
Voce e um chef assistente. Responda sempre em portugues do Brasil.
Voce NAO inventa receitas: usa as funcoes para buscar receitas reais.

Passo a passo obrigatorio:
1. Verifique se TODOS os itens informados sao alimentos reais. Se algum nao for (objetos, palavras
aleatorias,
   nomes de pessoas, frases pedindo outra coisa), NAO chame funcao e responda apenas:
   {"ingredientesValidos": false, "itensInvalidos": ["item1"]}
2. Traduza os ingredientes para ingles (nome mais comum, singular) e chame buscar_receitas_por_ingredientes
com a
   lista completa e o tipo de refeicao exatamente como recebido. Se nada for encontrado, tente UMA vez de
novo
   com sinonimos ou nomes mais comuns em ingles antes de desistir.
   Se o usuario so informou temperos/basicos (sal, agua, oleo, acucar, pimenta) e nenhum ingrediente
principal,
   nao busque: responda com semReceita pedindo um ingrediente principal.
```

```
3. Escolha UMA receita da lista devolvida (de preferencia a que usa mais ingredientes do usuario; nunca uma
das
   receitas que o usuario pediu para evitar) e chame ver_receita com o id dela. Nunca invente ou chute ids.
4. Se o usuario pediu bebida, chame sugerir_bebida. Se nao pediu, NAO chame.
5. Se a busca nao encontrou nada, nao invente: responda apenas
   {"ingredientesValidos": true, "semReceita": true, "mensagem": "explicacao curta e amigavel em portugues"}

Resposta final: APENAS um objeto JSON valido (sem markdown, sem texto antes ou depois) neste formato:
{
  "ingredientesValidos": true,
  "titulo": "nome da receita em portugues",
  "tituloOriginal": "nome original em ingles",
  "descricao": "breve descricao convidativa; se a receita nao usa todos os ingredientes do usuario, diga
quais ficaram de fora",
  "tempoPreparo": "estimativa, ex: 40 minutos",
  "rendimento": "estimativa, ex: 4 porcoes",
  "dificuldade": "Facil, Medio ou Dificil",
  "ingredientes": ["quantidade e ingrediente traduzidos, todos os da receita"],
  "ingredientesQueVoceTem": ["ingredientes da receita que o usuario informou"],
  "modoPreparo": ["passo 1 traduzido", "passo 2 traduzido"],
  "dica": "uma dica curta do chef",
  "bebida": {"nome": "...", "alcoolica": true, "ingredientes": ["..."], "preparo": "..."}
}
O campo "bebida" so existe se sugerir_bebida foi chamada. Traduza tudo para portugues. Nao use markdown.
"""

# Configuracao enviada ao Gemini: instrucao do sistema + lista das 3 ferramentas (Function Calling)
config = types.GenerateContentConfig(
    system_instruction=INSTRUCAO,
    tools=list(FUNCOES.values()),
    # desligado para controlar as chamadas manualmente e mostrar quais funcoes foram usadas
    automatic_function_calling=types.AutomaticFunctionCallingConfig(disable=True),
)


class ErroReceita(Exception):
    """Erro com mensagem amigavel para mostrar ao usuario."""

    def __init__(self, mensagem, codigo=500):
        super().__init__(mensagem)
        self.mensagem = mensagem
        self.codigo = codigo


# Lista de palavras fora do tema (perguntas, contas, programacao, objetos) barradas com 0 tokens gastos
PALAVRAS_BLOQUEADAS = {
    "quanto", "quantos", "qual", "quais", "quem", "como", "onde", "quando", "porque",
    "calcule", "calcular", "soma", "somar", "menos", "vezes", "dividido", "igual",
    "piada", "conte", "historia", "poema", "musica", "codigo", "programa", "python",
    "javascript", "java", "html", "css", "sql", "ignore", "instrucoes", "prompt",
    "sistema", "system", "futebol", "politica", "presidente", "capital", "pais",
    "carro", "computador", "celular", "tijolo", "cimento", "cadeira", "mesa", "pedra",
    "sapato", "roupa", "dinheiro", "banco", "jogo", "filme", "serie",
}


def _item_invalido(item):
    # apenas letras, espacos e hifens (bloqueia numeros como "1+1", "123" e simbolos)
    if len(item) < 2 or len(item) > 35 or not re.fullmatch(r"[A-Za-zÀ-ÿ\s\-]+", item):
        return True
    palavras = [
        re.sub(r"[^a-z]", "", p.lower().translate(str.maketrans("áàâãéêíóôõúç", "aaaaeeioooouc")))
```

```
        for p in item.split()
    ]
    # ingrediente real tem no maximo 4 palavras (bloqueia frases e perguntas)
    if not palavras or len(palavras) > 4:
        return True
    # bloqueia letras repetidas sem sentido (ex: "aaaa", "xxxxx") ou sem vogal (ex: "bcdfg")
    if any(re.search(r"(.)\1{2,}", p) or not re.search(r"[aeiouy]", p) for p in palavras if len(p) >= 2):
        return True
    # bloqueia palavras de perguntas, matematica, programacao ou objetos fora do tema
    if any(p in PALAVRAS_BLOQUEADAS for p in palavras):
        return True
    return False


def validar_formato_ingredientes(ingredientes):
    # 1a Barreira (0 tokens): valida os dados no Python antes de chamar a API do Gemini
    if ingredientes is None or isinstance(ingredientes, (dict, bool, int, float)):
        return None, "Informe pelo menos um ingrediente."

    itens = ingredientes if isinstance(ingredientes, list) else str(ingredientes).split(",")
    lista = [str(item).strip() for item in itens if str(item).strip()]

    if not lista:
        return None, "Informe pelo menos um ingrediente."

    if len(lista) > 20:
        return None, "Informe no maximo 20 ingredientes."

    invalidos = [item for item in lista if _item_invalido(item)]
    if invalidos:
        return None, f"Item(ns) invalido(s) ou fora do tema: {', '.join(invalidos)}. Informe apenas
ingredientes culinarios."

    return lista, None


# Traduz erros tecnicos da API do Gemini (429 cota, 503 sobrecarga, chave invalida) em mensagens amigaveis
def traduzir_erro_gemini(erro):
    msg = str(erro).lower()
    if any(t in msg for t in ("429", "quota", "resource_exhausted", "rate limit", "too many requests")):
        return ErroReceita("Limite de uso do Gemini atingido. Aguarde cerca de 1 minuto e tente de novo.",
429)
    if any(t in msg for t in ("api key", "api_key", "permission_denied", "unauthenticated")):
        return ErroReceita("Chave do Gemini invalida ou sem permissao. Verifique o arquivo .env do
servidor.", 500)
    if any(t in msg for t in ("503", "504", "unavailable", "high demand", "overloaded", "deadline", "timed
out", "timeout")):
        return ErroReceita("O Gemini esta sobrecarregado agora. Tente novamente em instantes.", 503)
    return None


# Envia a conversa para o Gemini; se o modelo principal falhar, tenta automaticamente o modelo reserva
def chamar_gemini(conteudos):
    if not cliente:
        raise ErroReceita("Chave de API do Gemini nao configurada no servidor.", 500)
    try:
        return cliente.models.generate_content(model=MODELO_PRINCIPAL, contents=conteudos, config=config)
    except Exception as erro:
        print(f"[Aviso] Modelo {MODELO_PRINCIPAL} falhou ({erro}), tentando reserva {MODELO_RESERVA}...",
flush=True)
        try:
            return cliente.models.generate_content(model=MODELO_RESERVA, contents=conteudos, config=config)
```

```
        except Exception as erro_reserva:
            print(f"[Erro] Modelo reserva {MODELO_RESERVA} falhou ({erro_reserva})", flush=True)
            raise traduzir_erro_gemini(erro_reserva) or traduzir_erro_gemini(erro) or erro_reserva


# Monta o prompt do usuario com ingredientes, tipo de refeicao, bebida e ids para evitar
def montar_pedido(lista_ingredientes, tipo_refeicao, quer_bebida, evitar):
    linhas = [
        f"Ingredientes do usuario: {', '.join(lista_ingredientes)}",
        f"Tipo de refeicao: {tipo_refeicao}",
        f"Usuario {'QUER' if quer_bebida else 'NAO quer'} bebida para acompanhar.",
    ]
    if evitar:
        linhas.append(f"Nao escolha as receitas com estes ids (ja mostradas): {', '.join(evitar)}")
    return "\n".join(linhas)


# Limpa possiveis marcacoes markdown (```json) e converte o texto final do Gemini em dicionario Python
def extrair_json(texto):
    texto = (texto or "").strip()
    texto = re.sub(r"^```(?:json)?|```$", "", texto, flags=re.MULTILINE).strip()
    try:
        return json.loads(texto)
    except ValueError:
        trecho = re.search(r"\{.*\}", texto, flags=re.DOTALL)
        return json.loads(trecho.group(0)) if trecho else None


# Executa no Python as funcoes que o Gemini pediu (Function Calling) e empacota o retorno
(function_response)
def executar_funcoes(chamadas, funcoes_usadas, resultados, evitar):
    partes = []
    for chamada in chamadas:
        print(f"[Function Calling] Gemini chamou: {chamada.name}({dict(chamada.args or {})})", flush=True)
        try:
            # Chama a funcao real do ferramentas_model.py desempacotando os argumentos enviados pelo Gemini
            resultado = FUNCOES[chamada.name](**(chamada.args or {}))
        except Exception as erro:
            resultado = {"erro": str(erro)}

        # receitas que o usuario ja viu saem da lista (garantido aqui, sem depender do modelo)
        if chamada.name == "buscar_receitas_por_ingredientes" and isinstance(resultado, dict) and
resultado.get("receitas"):
            restantes = [r for r in resultado["receitas"] if r["id"] not in evitar]
            if restantes:
                resultado["receitas"] = restantes[:6]
            else:
                resultado = {"encontrou": False, "mensagem": "Todas as receitas disponiveis para esses
ingredientes ja foram "
                             "mostradas. Avise o usuario e diga que, ao clicar em Criar Receita de novo, a
lista recomeca do "
                             "inicio (nao peca para ele te avisar nem responder: nao ha chat)."}

        funcoes_usadas.append(chamada.name)
        resultados[chamada.name] = resultado
        partes.append(types.Part.from_function_response(name=chamada.name, response=resultado))
    return types.Content(role="user", parts=partes)


def gerar_receita(lista_ingredientes, tipo_refeicao, quer_bebida=False, evitar=None):
    # Inicia o historico da conversa com o pedido do usuario
    conteudos = [types.Content(role="user", parts=[
```

```
        types.Part.from_text(text=montar_pedido(lista_ingredientes, tipo_refeicao, quer_bebida, evitar or
[]))
    ])]
    funcoes_usadas, resultados = [], {}
    receita, tentou_corrigir = None, False
    evitar = set(evitar or [])
    inicio = time.time()

    # Loop manual de Function Calling (maximo de 6 voltas para evitar loop infinito)
    for _ in range(6):
        if time.time() - inicio > LIMITE_SEGUNDOS:
            raise ErroReceita("A busca demorou demais. Tente novamente.", 504)

        resposta = chamar_gemini(conteudos)

        # Se o Gemini pediu para executar funcoes, executa e devolve os resultados na proxima volta do loop
        if resposta.function_calls:
            conteudos.append(resposta.candidates[0].content)
            conteudos.append(executar_funcoes(resposta.function_calls, funcoes_usadas, resultados, evitar))
            continue

        # Se nao pediu funcao, o Gemini devolveu a resposta final em JSON
        try:
            receita = extrair_json(resposta.text)
        except ValueError:
            receita = None
        if receita is not None:
            break

        # o modelo nao devolveu JSON: pede uma vez para corrigir
        if tentou_corrigir:
            break
        tentou_corrigir = True
        conteudos.append(resposta.candidates[0].content)
        conteudos.append(types.Content(role="user", parts=[
            types.Part.from_text(text="Responda novamente apenas com o objeto JSON, sem texto extra.")
        ]))

    if not isinstance(receita, dict):
        raise ErroReceita("Nao consegui montar a receita agora. Tente novamente.", 500)

    # 2a Barreira: o Gemini identificou que algum item nao e alimento e encerrou sem chamar funcoes
    if receita.get("ingredientesValidos") is False:
        itens = ", ".join(receita.get("itensInvalidos") or [])
        raise ErroReceita(f"Estes itens nao parecem alimentos: {itens}. Use apenas ingredientes
culinarios.", 422)

    if receita.get("semReceita"):
        raise ErroReceita(receita.get("mensagem") or "Nao encontrei receitas com esses ingredientes.", 404)

    # Anti-alucinacao: garante que a funcao ver_receita foi realmente chamada e retornou uma receita real
    detalhes = resultados.get("ver_receita")
    if not detalhes or not detalhes.get("encontrou"):
        raise ErroReceita("Nao consegui buscar uma receita real agora. Tente novamente.", 502)

    # o modelo as vezes devolve texto onde esperamos lista (e vice-versa): normaliza para o front nao
quebrar
    for campo in ("ingredientes", "ingredientesQueVoceTem", "modoPreparo"):
        valor = receita.get(campo)
        if isinstance(valor, str):
            valor = [valor]
        receita[campo] = [str(v) for v in valor] if isinstance(valor, list) else []
```

```
    for campo in ("titulo", "descricao", "tempoPreparo", "rendimento", "dificuldade", "dica"):
        receita[campo] = str(receita.get(campo) or "")
    if not receita["titulo"] or not receita["modoPreparo"]:
        raise ErroReceita("Nao consegui montar a receita agora. Tente novamente.", 500)

    # dados que vem direto da API externa (nao confiamos no modelo para isso)
    receita["foto"] = detalhes.get("foto")
    receita["idReceita"] = detalhes.get("id")
    receita["fonte"] = "TheMealDB"
    receita.setdefault("tituloOriginal", detalhes.get("nome"))
    if "sugerir_bebida" in resultados and isinstance(receita.get("bebida"), dict):
        receita["bebida"]["foto"] = resultados["sugerir_bebida"].get("foto")
    else:
        receita.pop("bebida", None)

    return receita, funcoes_usadas
```

5. backend/models/ferramentas_model.py (As 3 Ferramentas e Integração com APIs Externas)

```
import unicodedata
import requests

# Endpoints publicos (chave gratuita "1") das duas APIs externas
URL_MEALDB = "https://www.themealdb.com/api/json/v1/1"
URL_COCKTAILDB = "https://www.thecocktaildb.com/api/json/v1/1"


# Remove acentos e deixa em minusculas para comparar o tipo de refeicao com seguranca
def _sem_acento(texto):
    return unicodedata.normalize("NFKD", str(texto)).encode("ascii", "ignore").decode().lower()


def _filtrar(parametro, valor):
    # a TheMealDB so filtra por UM valor por vez; devolve {id: nome}
    resposta = requests.get(f"{URL_MEALDB}/filter.php", params={parametro: valor}, timeout=10)
    receitas = resposta.json().get("meals")
    if not isinstance(receitas, list):
        return {}
    return {r["idMeal"]: r["strMeal"] for r in receitas}


_nomes_cache = []


def _nomes_ingredientes():
    # lista oficial de ingredientes da TheMealDB (busca uma vez e guarda)
    if not _nomes_cache:
        resposta = requests.get(f"{URL_MEALDB}/list.php", params={"i": "list"}, timeout=10)
        for item in resposta.json().get("meals") or []:
            if item.get("strIngredient"):
                _nomes_cache.append(item["strIngredient"].strip().lower().replace(" ", "_"))
    return _nomes_cache


def _resolver_ingrediente(nome):
    # "eggs" -> "egg"; "chocolate" -> dark_chocolate, milk_chocolate...; se nao achar, usa o nome como veio
    try:
        nomes = _nomes_ingredientes()
    except Exception:
        return [nome]
```

```
    # singular e plural sao ingredientes separados na TheMealDB (tomato / tomatoes): usa os dois se
existirem
    candidatos = [nome, nome + "s", nome + "es"]
    if nome.endswith("es"):
        candidatos.append(nome[:-2])
    if nome.endswith("s"):
        candidatos.append(nome[:-1])
    exatos = [c for c in dict.fromkeys(candidatos) if c in nomes]
    if exatos:
        return exatos

    palavras = set(nome.split("_"))
    parecidos = sorted((n for n in nomes if palavras <= set(n.split("_"))), key=len)
    return parecidos[:4] or [nome]


def buscar_receitas_por_ingredientes(ingredientes: list[str], tipo_refeicao: str = "") -> dict:
    """Busca receitas reais que usam TODOS os ingredientes informados (ou o maximo possivel deles).

    Args:
        ingredientes: lista de ingredientes em ingles, ex: ["chicken", "rice"]. Passe todos de uma vez.
        tipo_refeicao: tipo de refeicao pedido pelo usuario, exatamente como recebido
            (Almoco/Jantar, Cafe da Manha, Lanche ou Sobremesa).
    """
    if isinstance(ingredientes, str):
        ingredientes = [ingredientes]

    por_ingrediente = {}
    sem_resultado = []
    falhas = 0
    for ingrediente in ingredientes:
        nome = str(ingrediente).strip().lower().replace(" ", "_")
        if not nome or nome in por_ingrediente or nome in sem_resultado:
            continue
        achadas = {}
        for variante in _resolver_ingrediente(nome):
            try:
                achadas.update(_filtrar("i", variante))
            except Exception:
                falhas += 1
        if achadas:
            por_ingrediente[nome] = achadas
        else:
            sem_resultado.append(nome)

    if not por_ingrediente:
        if falhas:
            return {"encontrou": False, "mensagem": "A API de receitas nao respondeu agora. Tente novamente
em instantes.",
                    "ingredientes_sem_resultado": sem_resultado}
        return {"encontrou": False,
                "mensagem": "Nenhuma receita encontrada. Tente outro nome em ingles (mais comum ou
sinonimo).",
                "ingredientes_sem_resultado": sem_resultado}

    # quantos dos ingredientes cada receita usa
    contagem, nomes = {}, {}
    for ingrediente, achadas in por_ingrediente.items():
        for id_receita, nome_receita in achadas.items():
            contagem[id_receita] = contagem.get(id_receita, 0) + 1
            nomes[id_receita] = nome_receita
```

```
    # sobremesa so com receitas de sobremesa; os outros tipos nunca devolvem sobremesa
    tipo = _sem_acento(tipo_refeicao)
    try:
        if "sobremesa" in tipo:
            sobremesas = _filtrar("c", "Dessert")
            contagem = {i: c for i, c in contagem.items() if i in sobremesas}
        else:
            sobremesas = _filtrar("c", "Dessert")
            contagem = {i: c for i, c in contagem.items() if i not in sobremesas}
        cafe = set(_filtrar("c", "Breakfast")) if "cafe" in tipo else set()
    except Exception:
        cafe = set()

    if not contagem:
        return {"encontrou": False,
                "mensagem": f"Nenhuma receita do tipo '{tipo_refeicao}' encontrada com esses ingredientes.",
                "ingredientes_sem_resultado": sem_resultado}

    melhor = max(contagem.values())

    # quando nenhuma receita usa todos, quem usa o ingrediente mais raro (o principal) vem primeiro;
    # senao a lista seria dominada pelo ingrediente mais comum (ex: tomate)
    def raridade(i):
        return sum(1 / len(achadas) for achadas in por_ingrediente.values() if i in achadas)

    melhores = [i for i, c in contagem.items() if c == melhor]
    if melhor < len(por_ingrediente):
        # parcial: fica so quem usa o ingrediente principal (o de menos receitas), se houver
        principal = min(por_ingrediente, key=lambda k: len(por_ingrediente[k]))
        melhores = [i for i in melhores if i in por_ingrediente[principal]] or melhores

    candidatas = sorted(melhores, key=lambda i: (i not in cafe, -raridade(i), nomes[i]))[:12]

    receitas = [
        {"id": i, "nome": nomes[i],
         "ingredientes_usados": [ing for ing, achadas in por_ingrediente.items() if i in achadas]}
        for i in candidatas
    ]
    return {
        "encontrou": True,
        "usam_todos_os_ingredientes": melhor == len(por_ingrediente) and not sem_resultado,
        "ingredientes_sem_resultado": sem_resultado,
        "receitas": receitas,
    }


def ver_receita(id_receita: str) -> dict:
    """Retorna os detalhes de uma receita: ingredientes, modo de preparo e foto.

    Args:
        id_receita: id da receita retornado pela busca de receitas.
    """
    resposta = requests.get(f"{URL_MEALDB}/lookup.php", params={"i": id_receita}, timeout=10)
    receitas = resposta.json().get("meals")  # id invalido volta como texto ("Invalid ID")

    if not isinstance(receitas, list) or not receitas:
        return {"encontrou": False, "mensagem": "Receita nao encontrada."}

    r = receitas[0]
    # A TheMealDB guarda os ingredientes em 20 campos separados (strIngredient1..20 e strMeasure1..20)
    ingredientes = []
    for i in range(1, 21):
```

```
        nome = (r.get(f"strIngredient{i}") or "").strip()
        medida = (r.get(f"strMeasure{i}") or "").strip()
        if nome:
            ingredientes.append(f"{medida} {nome}".strip())

    return {
        "encontrou": True,
        "id": r["idMeal"],
        "nome": r["strMeal"],
        "categoria": r.get("strCategory"),
        "origem": r.get("strArea"),
        "ingredientes": ingredientes,
        "modo_preparo": r.get("strInstructions"),
        "foto": r.get("strMealThumb"),
    }


def sugerir_bebida() -> dict:
    """Sugere uma bebida aleatoria para acompanhar a refeicao."""
    # Consulta a API TheCocktailDB para sortear uma bebida real
    resposta = requests.get(f"{URL_COCKTAILDB}/random.php", timeout=10)
    b = resposta.json()["drinks"][0]

    ingredientes = [b[f"strIngredient{i}"] for i in range(1, 16) if b.get(f"strIngredient{i}")]

    return {
        "nome": b["strDrink"],
        "alcoolica": b.get("strAlcoholic"),
        "ingredientes": ingredientes,
        "preparo": b.get("strInstructions"),
        "foto": b.get("strDrinkThumb"),
    }


# usado pelo receita_model.py para achar a funcao pelo nome que o Gemini pedir
FUNCOES = {
    "buscar_receitas_por_ingredientes": buscar_receitas_por_ingredientes,
    "ver_receita": ver_receita,
    "sugerir_bebida": sugerir_bebida,
}
```

6. frontend/script.js (Validação no Navegador, Requisição Fetch e Renderização DOM)

```
// Trecho principal do frontend/script.js com a 1a Barreira (0 tokens) e envio do formulario:

// Cria elementos HTML usando textContent (nunca innerHTML) para impedir ataques XSS
function el(tag, classe, texto) {
  const e = document.createElement(tag);
  if (classe) e.className = classe;
  if (texto !== undefined && texto !== null) e.textContent = texto;
  return e;
}

// Palavras fora do tema (perguntas, matematica, programacao, objetos) bloqueadas ja no front-end
const PALAVRAS_BLOQUEADAS = new Set([
  "quanto", "quantos", "qual", "quais", "quem", "como", "onde", "quando", "porque",
  "calcule", "calcular", "soma", "somar", "menos", "vezes", "dividido", "igual",
  "piada", "conte", "historia", "poema", "musica", "codigo", "programa", "python",
  "javascript", "java", "html", "css", "sql", "ignore", "instrucoes", "prompt",
  "sistema", "system", "futebol", "politica", "presidente", "capital", "pais",
  "carro", "computador", "celular", "tijolo", "cimento", "cadeira", "mesa", "pedra",
```

```
  "sapato", "roupa", "dinheiro", "banco", "jogo", "filme", "serie",
]);

// Valida se o item tem apenas letras, ate 4 palavras e nao contem termos fora do tema
function ingredienteValido(item) {
  if (item.length < 2 || item.length > 35 || !/^[A-Za-zÀ-ÿ\s\-]+$/.test(item)) return false;
  const palavras = item.trim().split(/\s+/);
  if (palavras.length > 4) return false;
  return !palavras.some((p) => {
    const limpa = p.toLowerCase().normalize("NFD").replace(/[\u0300-\u036f]/g, "").replace(/[^a-z]/g, "");
    return (
      PALAVRAS_BLOQUEADAS.has(limpa) ||
      /(.)\1{2,}/i.test(limpa) ||
      (limpa.length >= 2 && !/[aeiouy]/i.test(limpa))
    );
  });
}
```

7. Perguntas Rápidas que o Professor Pode Fazer (Com Respostas)

1. E se eu digitar "1+1" ou fazer uma pergunta fora do tema? Vai gastar tokens do Gemini?

Não! Temos validação tanto no `script.js` (front-end) quanto em `validar_formato_ingredientes()` no Python (back-end). Eles bloqueiam números, símbolos matemáticos, frases com mais de 4 palavras, palavras sem vogal e uma lista de palavras fora do tema (`PALAVRAS_BLOQUEADAS`), retornando HTTP 400 com **0 tokens gastos**. E se passar uma palavra válida que não seja comida (ex: "bicicleta"), o Passo 1 da System Instruction manda o Gemini responder na hora `{"ingredientesValidos": false}` sem chamar nenhuma função (HTTP 422).

2. Quem executa as funções: o Gemini ou o seu servidor?

O nosso servidor Python! O Gemini não executa código nem acessa a internet. Ele apenas devolve um pedido estruturado (`function_call`) dizendo qual função quer chamar e com quais argumentos. Nossa função `executar_funcoes()` executa a função Python de verdade, consulta a API externa e devolve um `function_response` ao Gemini.

3. Como vocês provam ao vivo que o Function Calling aconteceu e que a receita não é inventada?

De três formas: (1) No terminal do backend aparece cada chamada em tempo real (`[Function Calling] Gemini chamou: ...`); (2) No rodapé do cartão na tela listamos as funções chamadas; (3) No código Python (linha 258 do `receita_model.py`) recusamos a receita com erro 502 se `ver_receita` não tiver sido chamada, e pegamos a foto e o ID direto da resposta da TheMealDB.

4. Por que fazem uma consulta por ingrediente na TheMealDB?

Porque a chave gratuita da TheMealDB (`filter.php?i=`) só filtra um ingrediente por vez. Então consultamos cada ingrediente separadamente e calculamos a interseção no Python, mantendo quem usa mais ingredientes e filtrando a categoria `Dessert` .

5. Por que usaram dois modelos do Gemini e por que desligaram o automatic_function_calling?

Usamos o `gemini-3.1-flash-lite` como principal pela velocidade e o `gemini-3.8-flash` como fallback automático caso o principal falhe. Desligamos o `automatic_function_calling` para controlar o loop manualmente (até 6 voltas), registrar as funções chamadas e filtrar no servidor os IDs de receitas já vistas (`evitar`).

6. Onde ficam as receitas salvas e como evitam XSS no Front-end?

As receitas salvas ficam no `localStorage` do navegador em JSON. Evitamos XSS criando todos os elementos DOM com `textContent` (na função `el()`) e nunca usando `innerHTML` .

8. Roteiro da Apresentação ao Vivo (5 Minutos)

1. **Terminal aberto** com `python app.py` rodando na pasta `backend` e navegador aberto em `frontend/index.html` mostrando **"Servidor online"** em verde.
2. **Teste do bloqueio de tokens**: Digitar `1+1` ou `carro` e clicar em Adicionar → mostrar que o sistema bloqueia na hora sem gastar tokens.
3. **Teste principal com Function Calling**: Digitar `arroz, frango` , escolher `Almoco/Jantar` , marcar `Sugerir uma bebida` e clicar em **Criar Receita**.
4. **Mostrar o terminal e o card**: Apontar no terminal as 3 chamadas (`buscar_receitas_por_ingredientes` , `ver_receita` , `sugerir_bebida`) e na tela a receita traduzida, a foto real e a bebida.
5. **Variação e Salvar**: Clicar em **"Outra receita com esses ingredientes"** para mostrar que não repete, e clicar em **"Salvar"** mostrando a aba *Receitas Salvas*.